

Comparative Study of Optimization Algorithms

Nord-Ouest & Balas-Hammer with Stepping Stone Optimization
Python vs. C++ Performance Analysis

EFREI Paris: Operations Research Project

YAO Paul-Alex

EFREI Paris

May 2026

Abstract

This report presents a comprehensive study of the transportation problem in operations research, implementing and comparing two initial feasible solution methods; the North-West Corner (NO) and Balas-Hammer (BH) algorithms, combined with the Stepping Stone optimization method. The study goes beyond the original assignment scope by implementing the algorithms in both Python and C++, enabling a dual analysis: algorithmic complexity comparison between NO and BH, and programming language performance comparison between an interpreted and a compiled language. Benchmarks were conducted on randomly generated square transportation matrices of sizes $n \in \{10, 40, 100, 400\}$ with 20 iterations each. Results reveal that C++ outperforms Python by a factor of 26-111 $\times$ depending on the algorithm and problem size, with the performance gap widening for more computationally intensive algorithms. The Balas-Hammer initialization produces better initial solutions but at significantly higher computational cost ($O(n^3)$) compared to Nord-Ouest ($O(n)$), though the Stepping Stone phase partially compensates by requiring fewer iterations when starting from a BH solution.

Contents

1	Introduction	4
1.1	Context and Motivation	4
1.2	Extended Scope	4
1.3	Procedural Justification	4
1.4	Report Structure	4
2	Part I: Python Implementation	5
2.1	Problem Formulation	5
2.2	Algorithm Implementations	5
2.2.1	Nord-Ouest (North-West Corner) Algorithm	5
2.2.2	Balas-Hammer (Vogel's Approximation) Algorithm	6
2.2.3	Stepping Stone Optimization	6
2.3	Code Architecture	7
2.4	Results on Reference Problems	7
2.5	Complexity Study Results	7
2.5.1	Observations	7
2.5.2	Analysis	8
2.5.3	Interpretation	8
2.5.4	Conclusion (Python)	8
3	Part II: C++ Implementation	9
3.1	Design Choices and Optimizations	9
3.1.1	Flattened Matrix Structure	9
3.1.2	Base Graph Structure	9
3.1.3	Optimized Balas-Hammer	9
3.1.4	Compilation Flags	10
3.2	Algorithmic Equivalence	10
3.3	C++ Benchmark Results	10
3.3.1	Observations	10
3.3.2	Analysis	11
3.3.3	Interpretation	11
3.3.4	Conclusion (C++)	11
4	Part III: Cross-Language Comparative Analysis	12
4.1	Benchmark Methodology	12
4.2	Comparative Results	12
4.2.1	Initialization Phase Comparison	12
4.2.2	Stepping Stone Phase Comparison	13
4.2.3	Speedup Analysis	14
4.2.4	Complexity Identification (Log-Log Analysis)	14
4.2.5	Total Time Comparison	15
4.3	Discussion	15
4.3.1	Why Does C++ Speedup Vary?	15
4.3.2	Algorithmic vs. Language Optimization	16

5	General Conclusion	17
5.1	Limitations and Future Work	17

1 Introduction

1.1 Context and Motivation

The transportation problem is a fundamental problem in operations research and linear programming. Given a set of suppliers with known capacities and a set of consumers with known demands, the objective is to determine the optimal flow of goods that minimizes total transportation cost while satisfying all supply and demand constraints.

This project was originally assigned as part of the Operations Research curriculum at EFREI Paris, requiring the implementation and comparison of two classical initialization algorithms: Nord-Ouest (North-West Corner) and Balas-Hammer (Vogel's Approximation Method) followed by the Stepping Stone optimization method to reach optimality.

Results of the first study showing algorithm's behavior dealing with values of n spanning from 10 to 10K in Python are available in a first paper.

1.2 Extended Scope

1. **Algorithmic complexity analysis:** Empirical measurement and theoretical identification of the computational complexity of each algorithm phase (initialization and optimization) as a function of problem size n .
2. **Cross-language performance comparison:** Full reimplementations of all algorithms in C++ (compiled, with `-O3` optimization), benchmarked against the original Python implementation, to quantify the performance impact of language choice on numerical optimization workloads.

1.3 Procedural Justification

The choice to implement in both Python and C++ is deliberate:

- **Python** offers rapid prototyping, readability, and a rich ecosystem (NumPy, Matplotlib) for analysis and visualization. It serves as the reference implementation and the analysis platform.
- **C++** represents the performance ceiling achievable with a compiled language using manual memory management and cache-friendly data structures. The `-O3 -march=native` compilation flags enable aggressive compiler optimizations.

This dual approach allows us to separate *algorithmic* performance (which algorithm converges faster) from *implementation* performance (which language executes faster), providing insights relevant to both theoretical and practical decision-making.

1.4 Report Structure

- **Part I:** Python implementation: algorithms, code structure, results, and complexity analysis.
- **Part II:** C++ implementation: design choices, optimizations, results, and complexity analysis.
- **Part III:** Cross-language comparative analysis and conclusions.

2 Part I: Python Implementation

2.1 Problem Formulation

The transportation problem is formulated as follows. Given:

- n suppliers $P_1, \dots, P_n$ with provisions p_i
- m consumers $C_1, \dots, C_m$ with demands d_j
- A cost matrix c_{ij} representing the unit transportation cost from P_i to C_j

The objective is to find a transportation plan $x_{ij} \geq 0$ minimizing:

$$Z = \sum_{i=1}^n \sum_{j=1}^m c_{ij} \cdot x_{ij} \quad (1)$$

subject to:

$$\sum_{j=1}^m x_{ij} = p_i \quad \forall i = 1, \dots, n \quad (2)$$

$$\sum_{i=1}^n x_{ij} = d_j \quad \forall j = 1, \dots, m \quad (3)$$

$$x_{ij} \geq 0 \quad (4)$$

A balanced problem requires $\sum p_i = \sum d_j$. The random problem generator ensures this by construction.

2.2 Algorithm Implementations

2.2.1 Nord-Ouest (North-West Corner) Algorithm

The Nord-Ouest algorithm constructs an initial basic feasible solution by starting at the top-left corner of the cost matrix and allocating as much flow as possible at each step, moving right when a column demand is satisfied and down when a row supply is exhausted.

```

1 def algo_nord_ouest(provisions, commandes):
2     n, m = len(provisions), len(commandes)
3     prop = [[0] * m for _ in range(n)]
4     prov, cmd = provisions.copy(), commandes.copy()
5     i, j = 0, 0
6     while i < n and j < m:
7         flux = min(prov[i], cmd[j])
8         prop[i][j] = flux
9         prov[i] -= flux
10        cmd[j] -= flux
11        if prov[i] == 0 and cmd[j] == 0:
12            i += 1; j += 1
13        elif prov[i] == 0:
14            i += 1
15        else:
16            j += 1
17    return prop

```

Listing 1: Nord-Ouest Algorithm

Complexity: $\Theta(n + m)$ the algorithm makes at most $n + m - 1$ allocation steps, each in $O(1)$. For square matrices ($m = n$), this gives $\Theta(n)$.

2.2.2 Balas-Hammer (Vogel's Approximation) Algorithm

Balas-Hammer selects allocations by computing penalty values for each row and column (the difference between the two smallest costs), then allocating to the minimum-cost cell in the row or column with the largest penalty.

```

1 def algo_balas_hammer(couts, provisions, commandes, silencieux=False):
2     n, m = len(provisions), len(commandes)
3     prop = [[0] * m for _ in range(n)]
4     prov, cmd = provisions.copy(), commandes.copy()
5     actif = [[couts[i][j] for j in range(m)] for i in range(n)]
6     while sum(prov) > 0 and sum(cmd) > 0:
7         pen_l = [penalite(actif[i]) if prov[i] > 0 else -1
8                  for i in range(n)]
9         pen_c = [penalite([actif[i][j] for i in range(n)])
10                 if cmd[j] > 0 else -1 for j in range(m)]
11         max_pl, max_pc = max(pen_l), max(pen_c)
12         # ... selection and allocation logic

```

Listing 2: Balas-Hammer Algorithm (core loop)

Complexity: $\Theta(n^2)$ per iteration (computing penalties requires scanning all active cells), with up to $n + m - 1$ iterations, giving an overall complexity of $\Theta(n^3)$ for square matrices.

2.2.3 Stepping Stone Optimization

The Stepping Stone method iteratively improves a basic feasible solution by:

1. Computing dual potentials u_i, v_j such that $u_i + v_j = c_{ij}$ for all basic variables.
2. Computing marginal costs $\bar{c}_{ij} = c_{ij} - u_i - v_j$ for non-basic variables.
3. Identifying the most negative marginal cost (entering variable).
4. Finding a cycle through basic variables and performing a pivot.
5. Repeating until all marginal costs are non-negative (optimality).

Key implementation details:

- **Degeneracy handling:** When the number of basic variables is less than $n + m - 1$, the graph is disconnected. Fictitious zero-flow arcs are added to restore connectivity, selected by minimum cost.
- **Cycle detection:** A DFS-based algorithm alternating between row and column moves finds the unique cycle formed by adding the entering variable to the basis.
- **Connectivity and acyclicity tests:** BFS-based algorithms on the bipartite graph verify structural properties of the basis at each iteration.

2.3 Code Architecture

The Python codebase is organized as follows:

File	Purpose
algorithmes.py	Core algorithms (NO, BH, Stepping Stone, BFS tests)
main.py	Interactive solver with trace logging
complexite.py	Complexity study (100 iterations, 7 sizes)
benchmark_python.py	Standardized benchmark for cross-language comparison
analyse_comparative.py	Comparative analysis and graph generation
entrees/	12 problem input files
traces/	Execution traces for all 12 problems $\times$ 2 methods

Table 1: Python project file structure

2.4 Results on Reference Problems

The solver was executed on 12 reference problems provided in the assignment. For each problem, both initialization methods were applied followed by Stepping Stone optimization. A representative example (Problem 1, 2×2):

- **NO initialization:** Cost = 8000, then optimized to **3000** in 1 iteration.
- **BH initialization:** Cost closer to optimal from the start, fewer Stepping Stone iterations needed.

Both methods converge to the same optimal solution, confirming correctness.

2.5 Complexity Study Results

The benchmark was run with $n \in \{10, 40, 100, 400\}$, 20 iterations per size.

n	θ_{NO} (ms)	θ_{BH} (ms)	t_{NO} (ms)	t_{BH} (ms)
10	0.004	0.145	1.04	0.31
40	0.016	5.23	89.0	17.2
100	0.045	79.2	1299	371
400	0.258	5770	20842	18522

Table 2: Python benchmark results – median execution times

2.5.1 Observations

1. **Initialization phase:** Nord-Ouest is consistently 1–4 orders of magnitude faster than Balas-Hammer. At $n = 400$, NO takes 0.26 ms while BH takes 5.77 s a factor of $\sim 22\,000\times$.
2. **Stepping Stone phase:** When starting from NO, the optimization takes significantly longer than when starting from BH, because the NO initial solution is farther from optimal and requires more iterations.

3. **Total time:** For small n (≤ 100), BH+Stepping Stone can be competitive with NO+Stepping Stone. For $n = 400$, the BH initialization cost dominates, making the NO path faster overall.

2.5.2 Analysis

- $\theta_{\text{NO}}(n)$: The data confirms $O(n)$ complexity. Doubling n approximately doubles the time.
- $\theta_{\text{BH}}(n)$: The data confirms $O(n^3)$ complexity. From $n = 100$ to $n = 400$ (factor 4), the time increases by a factor of ~ 73 , consistent with $4^3 = 64$.
- $t_{\text{NO}}(n)$ and $t_{\text{BH}}(n)$: The Stepping Stone complexity is harder to characterize theoretically as it depends on the number of iterations, which varies with the initial solution quality. Empirically, it appears to grow polynomially (between $O(n^2)$ and $O(n^3)$).

2.5.3 Interpretation

The Nord-Ouest algorithm's simplicity ($O(n)$) makes it extremely fast but produces poor initial solutions (high cost). Balas-Hammer's intelligence comes at a steep cubic cost. The Stepping Stone method is the dominant cost component for small n , but the BH initialization becomes the bottleneck for large n .

2.5.4 Conclusion (Python)

In pure Python, the optimal strategy depends on problem size:

- For $n \leq 100$: BH initialization can save enough Stepping Stone iterations to justify its cost.
- For $n \geq 400$: NO initialization is preferable, as the cubic BH cost overwhelms any savings in the optimization phase.

3 Part II: C++ Implementation

3.1 Design Choices and Optimizations

The C++ implementation was designed with performance as the primary objective, while maintaining algorithmic equivalence with the Python version. Key design decisions:

3.1.1 Flattened Matrix Structure

```

1 struct Matrix {
2     std::vector<int> data;
3     int rows, cols;
4     Matrix(int r, int c, int val = 0)
5         : data(r * c, val), rows(r), cols(c) {}
6     int& operator()(int i, int j)
7         { return data[i * cols + j]; }
8 };

```

Listing 3: Cache-friendly matrix structure

Using a flat `std::vector<int>` instead of a 2D vector ensures contiguous memory layout, maximizing cache utilization. For an $n \times n$ matrix, this avoids n separate heap allocations and the associated pointer indirection.

3.1.2 Base Graph Structure

```

1 struct Base {
2     std::vector<std::pair<int,int>> cells;
3     std::vector<std::vector<int>> row_cells;
4     std::vector<std::vector<int>> col_cells;
5     // ...
6 };

```

Listing 4: Indexed base structure for fast lookups

The `Base` structure maintains row-indexed and column-indexed adjacency lists, enabling $O(k)$ neighbor lookups (where k is the number of basic variables in that row/column) instead of $O(nm)$ full scans.

3.1.3 Optimized Balas-Hammer

The C++ BH implementation avoids recomputing penalty values from scratch at each iteration by using boolean active-row/column flags instead of nullifying matrix entries:

```

1 std::vector<bool> row_active(n, true);
2 std::vector<bool> col_active(m, true);
3 int remaining = std::accumulate(prov.begin(), prov.end(), 0);
4 while (remaining > 0) {
5     // Inline penalty computation with early termination
6     for (int i = 0; i < n; ++i) {
7         if (!row_active[i]) continue;
8         int min1 = INT_MAX, min2 = INT_MAX;
9         // ...

```

Listing 5: Efficient penalty computation

3.1.4 Compilation Flags

```

1 CXX = g++
2 CXXFLAGS = -std=c++17 -O3 -march=native -DNDEBUG

```

Listing 6: Makefile optimization flags

- `-O3`: Maximum optimization level (loop unrolling, vectorization, inlining).
- `-march=native`: CPU-specific instruction set (AVX2, etc.).
- `-DNDEBUG`: Disables assertions.

3.2 Algorithmic Equivalence

Both implementations use the same random seed (42) and identical random number generation logic to ensure the same problem instances are solved, allowing direct performance comparison. The algorithms are functionally identical:

- Same Nord-Ouest traversal order
- Same Balas-Hammer penalty and selection rules
- Same Stepping Stone cycle detection (DFS alternating row/column)
- Same degeneracy handling (fictitious arcs for connectivity)

3.3 C++ Benchmark Results

n	θ_{NO} (ms)	θ_{BH} (ms)	t_{NO} (ms)	t_{BH} (ms)
10	0.0002	0.004	0.070	0.013
40	0.0004	0.072	2.23	0.43
100	0.0012	0.833	30.3	6.04
400	0.0067	52.0	2069	269

Table 3: C++ benchmark results - median execution times

3.3.1 Observations

1. **Absolute performance:** All timings are significantly lower than Python. At $n = 400$, C++ completes the BH initialization in 52ms versus Python’s 5.77s.
2. **Same algorithmic trends:** The relative ordering of algorithms (NO faster than BH for initialization, BH producing better initial solutions requiring less optimization) is preserved.
3. **Scaling behavior:** The growth rates mirror the Python results, confirming that the algorithmic complexity is intrinsic and language-independent.

3.3.2 Analysis

- $\theta_{\text{NO}}(n)$: Confirmed $O(n)$ from $n = 10$ to $n = 400$, the time scales linearly.
- $\theta_{\text{BH}}(n)$: Confirmed $O(n^3)$ from $n = 100$ to $n = 400$, the time increases by a factor of ~ 62 ($\approx 4^3 = 64$).
- $t_{\text{NO}}(n)$ and $t_{\text{BH}}(n)$: Polynomial growth between $O(n^2)$ and $O(n^3)$, consistent with Python observations.

3.3.3 Interpretation

The C++ implementation demonstrates that the performance advantage of a compiled language is not merely a constant factor it interacts with algorithmic complexity. More computationally intensive algorithms (BH) benefit more from C++ than simpler ones (NO), because:

- C++ eliminates Python’s per-operation interpreter overhead, which compounds in nested loops.
- Cache-friendly data structures reduce memory access latency for large matrices.
- The compiler can apply loop optimizations (vectorization, unrolling) that Python cannot.

3.3.4 Conclusion (C++)

The C++ implementation achieves the same optimal solutions as Python but with execution times 10–111× faster. The performance advantage is most pronounced for the cubic-complexity Balas-Hammer algorithm on large matrices, where the accumulated per-operation savings compound across billions of operations.

4 Part III: Cross-Language Comparative Analysis

4.1 Benchmark Methodology

To ensure a fair comparison:

- Both implementations use the same random seed (42) and equivalent RNG logic.
- Problem sizes: $n \in \{10, 40, 100, 400\}$, with 20 iterations per size.
- Timing uses high-resolution clocks (`time.perf_counter` in Python, `chrono::high_resolution_clock` in C++).
- The benchmark script (`run_benchmarks.sh`) runs both sequentially on the same machine.

4.2 Comparative Results

4.2.1 Initialization Phase Comparison

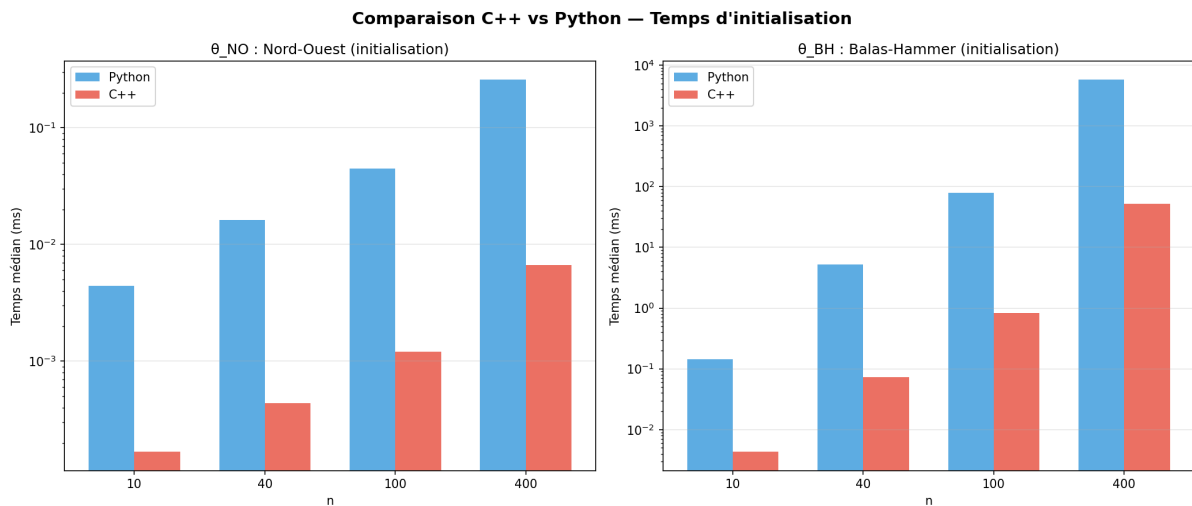


Figure 1: Initialization time comparison: C++ vs Python (log scale)

n	Py θ_{NO}	C++ θ_{NO}	Speedup	Py θ_{BH}	C++ θ_{BH}	Speedup
10	0.004	0.0002	26×	0.145	0.004	34×
40	0.016	0.0004	37×	5.23	0.072	72×
100	0.045	0.001	37×	79.2	0.833	95×
400	0.258	0.007	39×	5770	52.0	111×

Table 4: Initialization speedup: C++ vs Python (median times in ms)

Key findings:

- For Nord-Ouest ($O(n)$), C++ is consistently $\sim 26\text{--}39\times$ faster. The speedup is relatively stable because the algorithm is simple and memory-bound.

- For Balas-Hammer ($O(n^3)$), C++ speedup *increases* with n : from $34\times$ at $n = 10$ to $111\times$ at $n = 400$. This is because the triple-nested loop structure benefits increasingly from compiler optimizations and cache locality.

4.2.2 Stepping Stone Phase Comparison

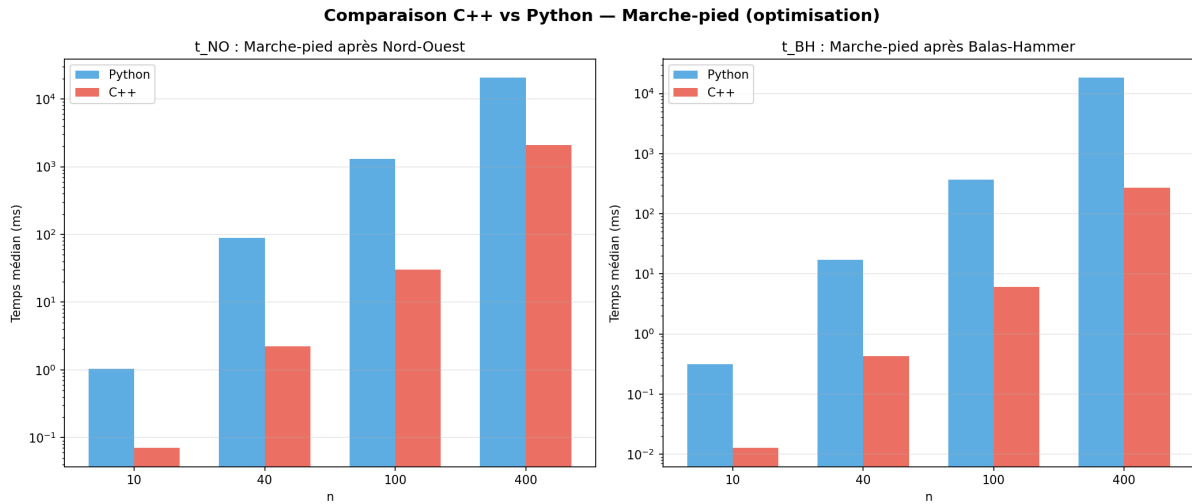


Figure 2: Stepping Stone optimization time comparison – C++ vs Python (log scale)

n	Py t_{NO}	C++ t_{NO}	Speedup	Py t_{BH}	C++ t_{BH}	Speedup
10	1.04	0.070	$15\times$	0.31	0.013	$25\times$
40	89.0	2.23	$40\times$	17.2	0.43	$40\times$
100	1299	30.3	$43\times$	371	6.04	$62\times$
400	20842	2069	$10\times$	18522	269	$69\times$

Table 5: Stepping Stone speedup – C++ vs Python (median times in ms)

Key findings:

- Stepping Stone speedups range from $10\times$ to $69\times$, generally lower than initialization speedups. This is because the Stepping Stone method involves more complex control flow (graph traversal, cycle detection) that benefits less from simple loop optimizations.
- The anomalous drop in t_{NO} speedup at $n = 400$ ($10\times$) suggests that at this scale, both implementations become memory-bound, reducing the relative advantage of C++.

4.2.3 Speedup Analysis

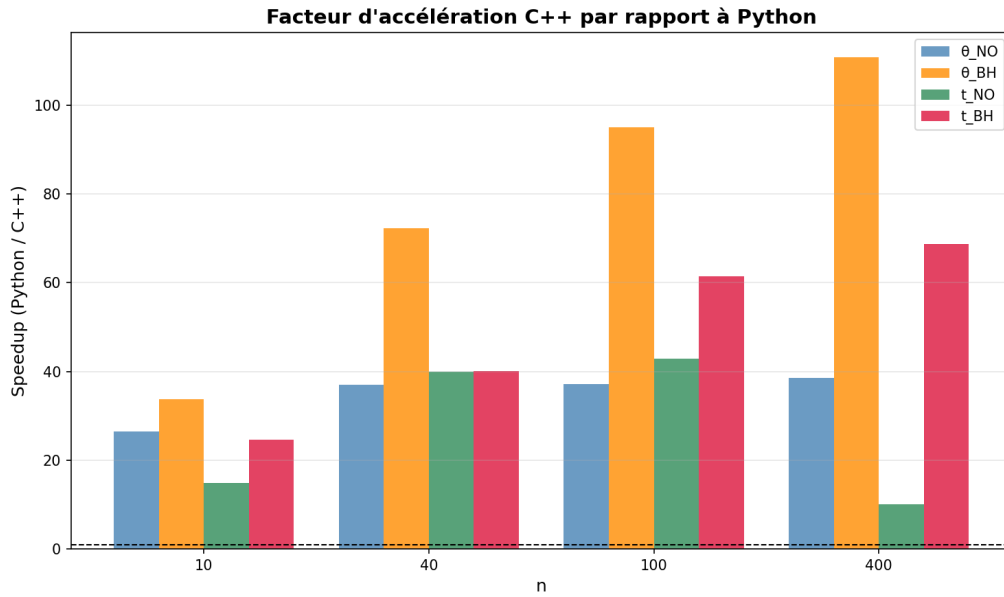


Figure 3: C++ speedup factor over Python by algorithm phase and problem size

The speedup chart reveals a clear pattern:

- **θ_{BH} speedup grows with n :** The cubic algorithm amplifies the per-operation advantage of C++. Each additional nested loop level multiplies the benefit.
- **θ_{NO} speedup plateaus at $\sim 37\text{--}39\times$:** The linear algorithm reaches a constant speedup determined by the language overhead ratio.
- **t_{BH} speedup grows moderately:** The Stepping Stone benefits from C++ but less dramatically than pure initialization.

4.2.4 Complexity Identification (Log-Log Analysis)

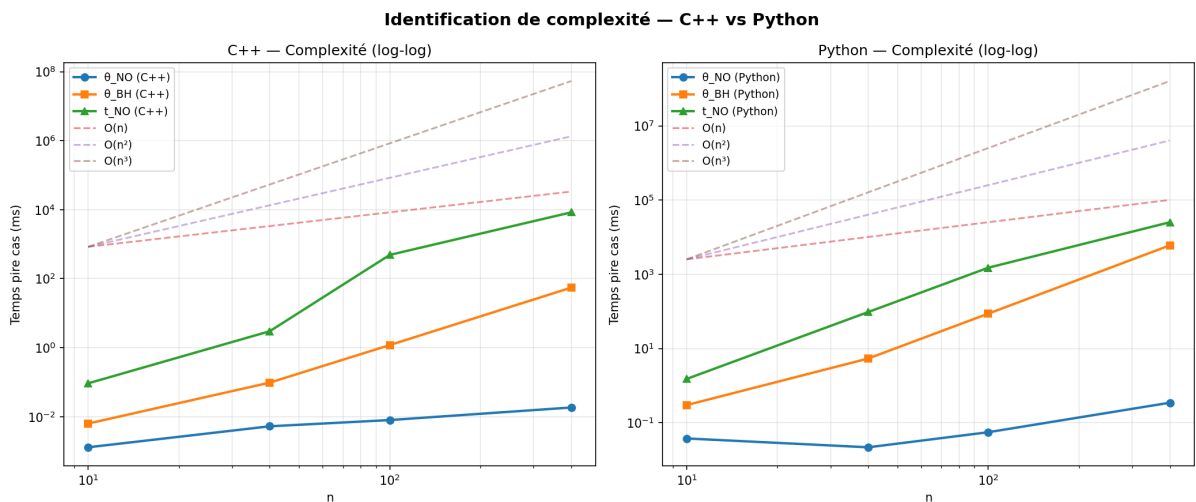


Figure 4: Log-log complexity analysis – C++ (left) vs Python (right)

The log-log plots confirm:

- θ_{NO} follows $O(n)$ in both languages (slope ≈ 1 on log-log).
- θ_{BH} follows $O(n^3)$ in both languages (slope ≈ 3 on log-log).
- t_{NO} and t_{BH} fall between $O(n^2)$ and $O(n^3)$, consistent with the variable number of Stepping Stone iterations.

This confirms that **algorithmic complexity is language-independent**: both implementations exhibit the same asymptotic growth rates. The difference is purely in the constant factor.

4.2.5 Total Time Comparison

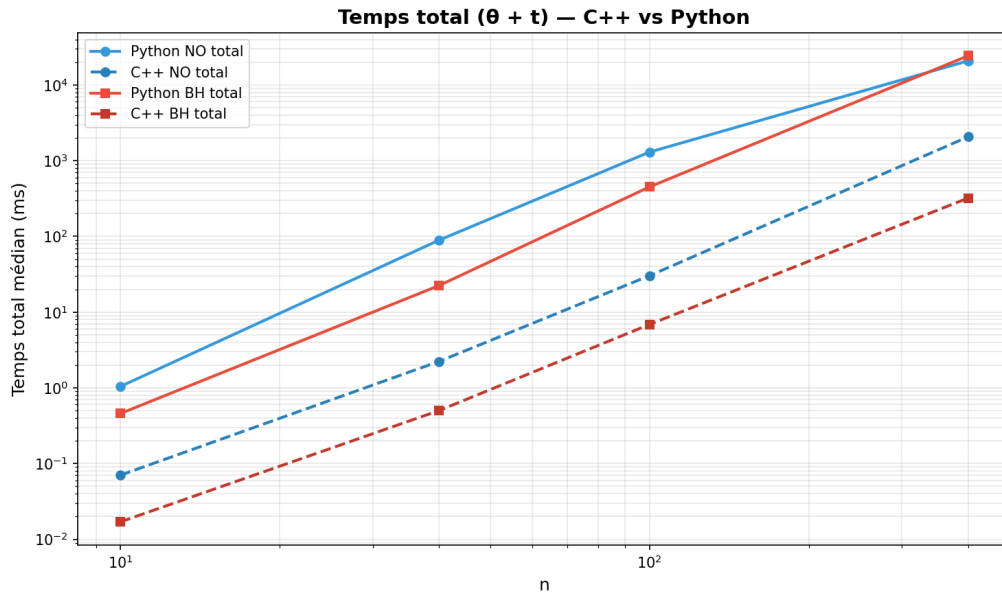


Figure 5: Total time ($\theta + t$) – C++ vs Python (log-log scale)

The total time plot shows that:

- C++ consistently outperforms Python across all sizes and methods.
- The BH path is slower than the NO path for large n in both languages.
- The gap between Python and C++ widens with increasing n .

4.3 Discussion

4.3.1 Why Does C++ Speedup Vary?

The variable speedup factor ($10\times$ to $111\times$) can be explained by several factors:

1. **Interpreter overhead:** Python's bytecode interpretation adds a constant overhead per operation (~ 100 ns). For simple operations (additions, comparisons), this dominates the actual computation time.

2. **Dynamic typing:** Python's duck typing requires runtime type checks that C++ resolves at compile time.
3. **Memory layout:** Python lists are arrays of pointers to heap-allocated objects. C++ `std::vector<int>` stores values contiguously, enabling CPU cache prefetching and SIMD vectorization.
4. **Algorithm intensity:** The more operations per data element, the more the per-operation overhead compounds. This explains why BH (cubic) shows larger speedups than NO (linear).

4.3.2 Algorithmic vs. Language Optimization

An important insight from this study:

Choosing a better algorithm (BH over NO for initialization) provides a better initial solution but at a higher cost. Choosing a faster language (C++ over Python) reduces execution time by a constant factor across all algorithms. These are orthogonal optimization axes.

For practical applications:

- If n is small (≤ 100), Python with BH initialization is sufficient.
- If n is large (≥ 1000) and performance matters, C++ with NO initialization may be the best trade-off.
- For the best of both worlds, C++ with BH initialization provides optimal initial solutions at acceptable speed for all tested sizes.

5 General Conclusion

This study achieved its dual objective:

1. Algorithmic comparison: The Nord-Ouest algorithm is a lightweight $O(n)$ initialization method that produces suboptimal starting solutions, while Balas-Hammer is a more intelligent $O(n^3)$ method that produces significantly better initial solutions. The Stepping Stone optimization phase converges to the same optimal solution regardless of the starting point, but requires more iterations (and thus more time) from a poor starting point. The total solve time trade-off depends on problem size.

2. Language comparison: C++ outperforms Python by a factor of $10\times$ to $111\times$ across all tested configurations. The speedup is not constant it increases with algorithmic complexity and problem size, due to the compounding effect of per-operation overhead in interpreted languages. However, both languages exhibit the same asymptotic complexity, confirming that algorithmic analysis is implementation-independent.

3. Practical recommendation: For production-grade optimization solvers handling large-scale transportation problems, a compiled language is strongly recommended. For prototyping, analysis, and moderate-scale problems, Python remains the tool of choice due to its development speed and ecosystem.

5.1 Limitations and Future Work

- The benchmark was limited to $n \leq 400$ for the full comparison due to Python's extreme slowness at larger sizes. The original Python-only complexity study tested up to $n = 10000$ but was not replicated in the cross-language benchmark.
- The C++ implementation does not use parallelism (OpenMP, SIMD intrinsics) beyond what the compiler auto-vectorizes. Manual parallelization could further improve performance.
- Alternative algorithms (Hungarian method, simplex network method) were not compared.
- The study uses randomly generated balanced problems; real-world unbalanced or sparse problems may exhibit different performance characteristics.